

AWS Essentials SDK

Setup and Configuration Guide

Revision (Undefined variable: build.PACKAGE_VERSION) Build (Undefined variable: build.EMIT_BUILD_BUILDNUM)

Table of Contents

Chapter 1: Introduction	5
Overview of the AWS Essentials SDK	5
Configuration Root	6
Configuration Master	6
Puppet Agent AMIs	6
How to Read This Guide	6
Chapter 2: Configuration Roots	8
Create S3 Buckets	8
Naming	9
Access Control	9
Create a Security Group	10
Create a Key Pair	10
Create an IAM Role	10
Create the Config-Root Instance	11
Install the S3 Command-Line Client	11
Create Directories	11
Install Scripts	11
Local and Remote Logging	12
Stop the Config-Root Instance	12
Configuration Root Account Management	13
Sharing S3 Buckets	13
Disaster Recovery	13
Security Recovery	14
Chapter 3: Configuration Masters	15
Create Security Groups	15
Configuration Agent Security Group	16
Configuration Master Security Group	16
Configuration Setup Security Group	17
Create a Key Pair	17
Create an IAM Role	17
Create an Elastic IP	18
Plan for VPC Topology	18
Create the Config-Master Instance	19
Install Software	19
Set S3 Buckets	20
Generate a GNU Privacy Guard (GPG) Key	20

Configure HTTPd	21
Configure the Puppet Master Server	21
Add Required Puppet Modules	21
Add a Cronjob	22
Install ENC-Common	22
Configure the Hostname (Optional)	22
Configure Remote Logging (Optional)	22
Final Steps	23
Conclusion	23
Chapter 4: Puppet Agent AMI Creation	24
Launch an Instance	24
Install Scripts	24
Local and Remote Logging	25
Update System Configuration	26
Install Puppet Agent	26
Disable Services	27
Configure Repositories	27
Final Preparation	27
Create the AMI	28
Using the AMI	28
User Data	28
IAM Role	28
Updating an AMI	29
Chapter 5: Application Installation	30
Configuration Root Installation	30
Configuration Master Installation	30
Chapter 6: System Validation	32
Chapter 7: Application Developer Information	33
Application Creation	33
Application Override Management	35

Chapter 8: Conclusion	37
Chapter 9: Appendix A: Glossary	38
Chapter 10: Appendix B: Bastion Hosts	40
Chapter 11: Appendix C: Auto Scaling Groups	41
Creating the Launch Configuration	41
Creating the Auto Scaling Group	42
Auto Scaling Features	42
Chapter 12: Appendix D: AWS Essentials Management	43
Best Practices	43
Instance Overview	43
Type Guidelines	44
Application Responsibilities	45
Logging	45
Startup	45
Chapter 13: Glossary	46

Chapter 1: Introduction

Welcome to the AWS Essentials SDK Setup and Configuration guide. The AWS Essentials SDK includes this guide to enable you to set up an application server and configuration system on Amazon Web Services (AWS) EC2 instances. Many PESDCA platforms and services require this setup.

Puppet software is used to create Puppet Master servers and provision and configure Puppet Agent nodes. Both the Puppet Master servers and the Puppet Agent nodes are AWS EC2 instances. Puppet Agent instances choose all software packages to install at the time of launch. For more information, see the Puppet Labs Web site:

<https://docs.puppetlabs.com>

This guide contains instructions on setting up AWS and using the scripts and tools included with this product. In addition, it provides information for application developers who want to create applications that can be versioned, installed, and configured through this setup.

This guide assumes a high degree of AWS knowledge. AWS provides extensive documentation for its services and an excellent search engine. Refer to AWS documentation if you are having difficulty with any of the steps that require using the AWS Web console or application programming interfaces (APIs). For additional help, the EMIT Networking Consortium (ENC) provides a page with links to important AWS information:

https://emit-networking.org/en-us/techservices/Pages/aws_help.aspx

In addition to AWS knowledge, a basic understanding of Puppet architecture and terms is recommended for a full understanding of this application server and configuration system setup. This understanding is especially helpful for managing the system.

Overview of the AWS Essentials SDK

The configuration system described in this guide and included in the AWS Essentials SDK system comprises three main components:

1. **Configuration root.** An optional global application repository that can be accessed from multiple accounts and regions. It stores applications that can be automatically copied to one or more configuration masters.
2. **Configuration master.** The Puppet Master server instance that hosts applications (through http and Puppet) for Puppet Agent instances.
3. **Puppet Agent Amazon Machine Images (AMIs).** AMIs configured to access specific configuration masters within the same account and region.

Note: Setting up a bastion host is recommended in highly secure systems. See Appendix B: Bastion Hosts.

Configuration Root

A configuration root acts as a repository for shared applications. Every configuration master that is given read access has access to the applications and configuration information stored on the root. A configuration root consists of the following:

- One or more S3 buckets that contain applications and other configuration files
- An AWS EC2 instance used to manage the S3 buckets

The configuration root is optional, because applications can also be stored and managed directly on a configuration master; however, using a configuration root is the only way to configure the system to work in multiple regions and for multiple accounts. If your organization requires synchronizing applications across multiple regions or accounts, you must set up a configuration root.

Configuration Master

The configuration master is an EC2 instance that is a Puppet Master server. At least one configuration master must be installed in each account, region, and VPC that needs to use the application server and configuration system. Configuration masters cannot be shared between accounts, regions, or VPCs due to limitations in AWS security and firewalls.

The configuration master must also be a Hypertext Transfer Protocol (HTTP) server. Puppet Agent instances access the software repositories through HTTP.

If a configuration master has access to one or more configuration roots, it downloads initial information about applications and configuration from these roots to its repository. Any application and configuration information stored on the configuration master itself is then added to the repository. Locally stored applications and configurations override configuration root information if both contain versions of the same application or configuration information.

More than one configuration master can be created in an account and region, and it may be beneficial to create multiple configuration masters for specific uses. For example, you may want a dedicated configuration master to store and test software under development before incorporating it into a production system.

Puppet Agent AMIs

When these AMIs are launched, they run the Puppet Agent to install software packages stored on a configuration master. The software to install on the instance is determined at time of launch by setting user data.

How to Read This Guide

This guide provides multiple entry points for different audiences:

- Installers who are setting up both a configuration root and configuration master need to follow all instructions in the chapters from *Configuration Roots* through *System Validation*.
- Installers who are setting up only a configuration root need the instructions in the following chapters:
 - *Configuration Roots*
 - *Application Installation*
- Installers who are setting up only a configuration master can begin at the *Configuration Masters* chapter and continue through *System Validation*.
- The information in the *Application Developer Information* chapter is for application developers, although developers may also need to install AWS Essentials before using it as a development platform, in which case the chapters for installers are also needed.
- Refer to the glossary for a specific definition of commonly used AWS terms.
- Other appendices are referenced in the guide, where appropriate.

Chapter 2: Configuration Roots

As mentioned previously, a configuration root consists of the following:

- One or more S3 buckets that contain applications and other configuration files
- An AWS EC2 instance used to manage the S3 bucket, which will be called the “config-root instance” in the remainder of this guide

This section describes how to create a configuration root. Individuals who will not be using a configuration root or who will be accessing preexisting, shared configuration roots can skip this section.

It is essential to tightly manage access to the configuration root. The config-root instance has write privileges to the S3 application repository; thus, it is very powerful and essentially provides root access to all programs and configuration, potentially across multiple accounts and regions. In addition, the owner of the account where the S3 bucket was created controls access and permissions for the S3 bucket.

It is recommended that a configuration root be isolated in its own account—that is, in an account without any configuration masters, other EC2 instances, or other AWS components or services. Isolating the configuration root in its own account, while not required, provides security partitioning, making it easier to control access. If you choose not to isolate the configuration root in its own account, precise management is required to ensure no unauthorized access.

The config-root instance runs in the specific region you select, which is referred to in this guide as the root region. If a bastion host is being used (see Appendix B: Bastion Hosts), it should likely be in the root region. A bastion host also provides an alternative to AWS IAM for controlling access to the config-root instance.

Setting up AWS services and creating the configuration root involves creating the following:

- S3
- A security group
- A key pair
- An IAM role
- The config-root instance

Create S3 Buckets

Because one of the main components of a configuration root is an S3 bucket where applications are stored, every configuration root must have at least one S3 bucket. In many cases, multiple buckets are useful. For example, an organization may have one set of applications they wish to share publicly and another that they wish to remain private. Creating separate S3 buckets for different application sets allows you to make different application sets available to different groups.

Each config-root bucket contains current versions of applications and configuration information that are downloaded to each configuration master given access.

Follow AWS instructions to create one or more buckets. Guidelines are given below.

Naming

S3 buckets must have unique names. AWS enforces this unique naming. You may need to experiment to come up with a naming scheme for your S3 buckets. The configuration system itself has no naming requirements. The remainder of this guide will refer generically to a configuration root bucket as the “config-root” bucket; however, you should replace “config-root” with the actual name of your bucket when following instructions.

Access Control

By default, when an S3 bucket is first created, all access to the bucket is limited to the bucket’s owner. Access should be managed according to the following guidelines.

Write Access

Permission to write files to the buckets must be strictly controlled, and should be considered the same as root access for the entire configuration system. During this setup, you will create an IAM role that has write access to the config-root buckets, and the config-root instance will be assigned that role. This means that after finishing the setup process in this guide, write access to the S3 bucket is limited to the following:

- The bucket’s owner (through the AWS Web console)
- The config-root instance

Typically, for the strongest security, it is recommended that you not extend write access beyond these two entities. Write access can then be limited to users who are given access to the EC2 instance.

Warning: Through the configuration system, applications stored in S3 buckets are installed and run as root on every instance in the system that accesses them. This could provide a powerful point of attack on the entire system for anyone with malicious intent. To reiterate: permission to write files to the buckets must be strictly controlled.

Read Access

All configuration masters that you want to allow to use the application set in a bucket must be given read access to that bucket. The buckets should contain no passwords or other secrets, so that read access can be freely granted. Application developers should ensure that this remains true for their applications, and your organization should avoid manually adding such information.

Depending on the access that you grant worldwide, you may need to consider export control laws regarding software that you place in a bucket.

Logging

S3 allows for logging, which may be valuable to set up for this critical resource.

Create a Security Group

The purpose of this dedicated security group is to control access to the config-root instance from outside of AWS. Create this security group in the root region. The security group must provide SSH access, but should be limited to the external IP range of the group that manages the configuration root. Typically, you should not need to open any other ports. Restrict the firewall rules for this group as much as possible.

Note: If a bastion host is used, SSH access should be limited to the bastion.

Create a Key Pair

This key pair will provide root access to the config-root instance. Create it in the root region. Store the private key securely.

Warning: Because the key pair provides root access to the config-root instance, it also provides write access to the S3 bucket.

Note: If a bastion host is being used, the private key may only need to be placed on the bastion and backed up securely.

Create an IAM Role

IAM roles allow permissions to be assigned to running instances without distributing secret keys to those instances. The config-root instance must have an assigned role that grants it very powerful access to the S3 buckets.

Perform the following:

1. Create an IAM role on the configuration root's account.
2. Generate a custom policy, giving it a logical name that you can remember.
3. Copy and paste the following into your policy, replacing <config-root name> with the name of one of your S3 buckets. You can repeat the last line with different bucket names to provide access to multiple S3 buckets.

```
{
  "Statement": [
    {
      "Action": [
        "s3:*"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:s3:::<config-root name>/*"
      ]
    }
  ]
}
```

}

Warning: Any IAM user on the account who has permission to create an EC2 instance and attach this IAM role to it will be able to create instances that have write access to the S3 bucket.

Create the Config-Root Instance

Everything is now in place to create the config-root instance itself in the root region. Launch an EC2 instance and ensure that you set the following:

- Amazon Linux operating system
- Assign the IAM role created for the config-root instance
- Assign the security group created for the config-root instance
- Assign the key pair created for the config-root instance
- Select a persistent file system

Install the S3 Command-Line Client

The config-root instance requires the S3 command-line client. The config-root instance requires the S3 command-line client. To install this client, go to <http://s3tools.org/repositories> and install the correct s3tools.repo file on your config-root instance. Then execute the following:

```
yum install s3cmd
```

Create Directories

For every S3 bucket you created to hold application sets, you must create a directory with a matching name in the \$HOME directory of the ec2-user. The S3 buckets will be managed by changing content in these directories.

Install Scripts

The AWS Essentials SDK includes a script that needs to be installed and run on each boot of a configuration root instance. The necessary files are located in the config-root directory. Follow these steps to properly install this script:

1. Copy the following scripts and files to a temporary location on the instance:
enc-root-bin
enc-root-sysconfig
2. Log in to the instance as the ec2-user, then open a root shell using sudo.
3. Rename the scripts and move them to the correct directories.

```
mv enc-root-sysconfig /etc/sysconfig/enc-root
mv enc-root-bin /usr/bin/enc-root
```

4. Ensure that the scripts have the correct ownership and permissions.

```
chown root:root /etc/sysconfig/enc-root
```

```
chown root:root /usr/bin/enc-root
chmod 0644 /etc/sysconfig/enc-root
chmod 0744 /usr/bin/enc-root
```

5. Configure the service to start automatically on boot by adding the following line to `/etc/rc.local`:

```
/usr/bin/enc-root
```

6. Edit the `/etc/sysconfig/enc-root` file and set the appropriate value for `HOSTNAME`.

Local and Remote Logging

This section contains important information about local and remote logging and the steps necessary to configure it. Remote logging is a key aspect of this configuration system. Instances may start and be configured at almost any time, and managing the logs across all of the instances would be very difficult without a centralized logging system.

The scripts installed in the previous section configure instances to use Papertrail for log centralization. Papertrail was chosen because of its cost, features, and Web-based access. For small, simple systems, a dedicated, centralized syslog host may be sufficient.

Papertrail creates standard syslog-compatible Log Destinations which are effectively a hostname and port combination, along with a setting that allows new systems to be automatically added. The hostname and port are used as part of the agent configuration.

A full description of using Papertrail is outside the scope of this document.

Papertrail is part of the solution for remote monitoring. In addition to logging, PagerDuty was chosen to provide alerting and incident management. The configuration and management of PagerDuty is outside the scope of this document.

Note: The specific host and port used by Papertrail is included in the `enc-root` `sysconfig` file. The default values disable remote logging.

Follow these steps to configure remote logging:

1. Edit the `/etc/sysconfig/enc-root` file.
2. Update the values of `LOGGING_HOST` and `LOGGING_PORT`.
3. Review the other settings and make appropriate changes based on operating system and application needs.
4. Save the file.

Stop the Config-Root Instance

At this point, you have completed the base installation of the configuration root. The `config-root` instance can be stopped. It is a good practice to stop the `config-root` instance whenever you are not performing any management tasks (described in the next section). This provides both cost savings and additional security.

Note that at this point there are no applications shared by the bucket. Adding applications is described in the Application Installation section.

Configuration Root Account Management

Once the account has been set up, it must be managed. Management at the account level includes loading application files and synchronizing them with the S3 buckets.

All account management requires access to the config-root instance and its key pair or access through the bastion host. The config-root instance is normally stopped, and so it should be started in order to manage the account. When management tasks are complete, the instance should be stopped again.

Warning: Those with the ability to manage the account effectively have root access on all systems and applications in the account. This ability must be carefully guarded.

Sharing S3 Buckets

The S3 buckets associated with a configuration root can be shared (read-only) with other accounts so that configuration masters can share the applications in them. The configuration root account owner uses the AWS management console to share the buckets.

Both the bucket and the files need permissions granted. For the bucket, select the bucket and add the List and View permissions for the email address of the account that will have access. For each file, grant the Open/Download and View permissions (using the email address or the drop-down list).

The buckets do not become visible in the other accounts' AWS management consoles, but programmatic access (and several third-party tools) will show the bucket and contents.

Sharing a bucket as described provides an account with read access to everything in the bucket. Permission cannot be granted to view only some of the contents of a bucket. Instead, use more than one S3 bucket to control access to different application sets.

Disaster Recovery

The process described here downloads all persistent information from S3 buckets into a new config-root instance to completely restore the config-root instance from scratch:

1. Create a new config-root instance (following the instructions in this guide), but using the existing buckets, role, security group, and key pair.
2. Copy all application files from the S3 bucket to the config-root directories.

Security Recovery

A loss of security (either through loss of control of the config-root instance key pair or unauthorized access to the config-root instance or buckets) is extremely difficult to recover from. Obviously, it should be avoided by strictly controlling access to these resources.

The danger of losing control is unauthorized modification of the S3 bucket contents, which causes indirect loss of control of any instance running in the configuration system. Modification of the S3 files allows changes to be propagated to all configuration masters and their instances. This means a loss of control can extend beyond the account and make restoring security a configuration-system-wide issue, rather than just an account-wide issue for the configuration root.

S3 offers versioning and logging which can be used to track access to S3 items and recover previous versions. Together, these could allow for recovery from malicious or accidental modifications. Otherwise, a complete inspection of the files in S3 would be required in order to ensure their integrity.

If malicious modifications to the contents of S3 are found, then the effects of that need to be mitigated. Usually this would include a top-down reset of the systems. If the specific modifications are known then more precise repairs are possible. If not, then the following process should ensure a clean restart.

1. Repair any damage to the S3 bucket. Ensure access to the bucket is secured to the same level as before (limiting write access only to the config-root instance).
2. Verify that the security groups and roles are correct in each region.
3. Depending on the extent of damage, terminate and recreate the config-root instance. You may need to create a new key pair for the instance.
4. Terminate all configuration masters in the configuration system. These may be on multiple accounts.
5. Create new configuration masters and synchronize them with the new config-root instance.
6. Recreate all affected instances, generally by launching and configuring new instances from the base Puppet Agent AMIs. Depending on the extent of the damage the old instances may need to all be terminated at the same time. This would apply if the damage introduced some system-wide collaborative software—difficult, but possible.

Chapter 3: Configuration Masters

The configuration master is an EC2 instance that is a Puppet Master server and (inside a VPC) a Port Address Translator (PAT). At least one configuration master must be installed in each account, region, and VPC that needs to use the application server and configuration system. Configuration masters are the true workhorses of the AWS Essentials configuration system and can operate with or without a configuration root.

This section describes how to set up a configuration master. The process of setting up a configuration master is virtually identical, whether or not you are using a configuration root, with the exception that two additional steps are required to use a configuration root. These steps are identified in the instructions.

Security is a key design consideration for the configuration master, as an individual with root access could modify the deployment rules or software deployed to other instances. The configuration master is set up with root access for installing software on all instances that use that master.

The setup described in this section provides the following layers of security:

- The configuration master is placed into two security groups that exist only for configuration.
- Internal access to the configuration master is restricted to http and puppet.

You should ensure that all external access to the configuration master is also strictly controlled.

The AWS account portion of the setup requires creating the following:

- Security groups
- A key pair
- An IAM role
- An Elastic IP

The EC2 instance that becomes the configuration master is referred to as the config-master instance in the remainder of this guide.

As mentioned in the introduction, there are good use cases for having more than one config-master instance in the same region and account. When setting up additional config-master instances in a region, they may use the same security groups and IAM role, but each config-master instance must have its own key pair and Elastic IP.

Create Security Groups

The configuration master requires two security groups. A third security group should also be created that will be used when creating new Puppet Agent AMIs associated with a configuration master. Every account and region that contains at least one configuration master needs these three security groups.

If the configuration master is located inside a VPC, then VPC security groups must be created. VPC security groups allow configuration for both inbound and outbound traffic. This section describes the settings required for inbound traffic, and assumes that all outbound traffic is permitted (the default). This should be acceptable for most installations. This guide also points out some important differences for the necessary inbound traffic rules for VPC installations. For more information on setting up VPC security groups, refer to the Amazon documentation.

Configuration Agent Security Group

This security group provides internal AWS access to the config-master instance for Puppet Agent instances in the configuration master's region (or VPC). This group is referred to as the config-agent group in the remainder of this guide.

Create this group. There are no inbound rules to assign to this group. If the associated config-master instance will be in a VPC, then this group needs to be created in the same VPC.

Configuration Master Security Group

This security group provides SSH access to the config-master instance from outside AWS. The security group must provide SSH access, but should be limited to the external IP range of the group that manages the config-master instance. This group also provides access to puppet for members of the config-agent group. This guide refers to this security group as the config-master group.

If the config-master will be in a VPC, then this group should be created inside the same VPC.

Create this group and then create the following input rules:

- Allow SSH access for the appropriate external IP range
- If the security group is inside a VPC, allow all traffic (TCP, UDP, ICMP) from members of the config-agent group
- If the security group is outside a VPC, create the following rules:
 - Allow HTTP access to members of the config-agent group
 - Allow TCP through port 8140 for members of the config-agent group

Note: If a bastion host is being used, then SSH access should be limited to the bastion host security group (in the bastion's region) or to the bastion Elastic IP address (in other regions).

Configuration Setup Security Group

This security group provides SSH access outside AWS for creating Puppet Agent AMIs. Creating these AMIs is described later, but since they will all require the same security group, create that security group now as part of the configuration master setup. The security group is used only when creating the instance that you will save as an AMI. After the AMI has been saved from the instance, the security group is no longer used by new instances launched from the AMI.

Create this group and allow SSH access for the IP ranges of groups you wish to be able to create Puppet Agent AMIs. This may be identical to the config-master group, but it is complying with the AWS best practice of not reusing security information for multiple purposes. This enables you to disable SSH access for this group whenever you are not directly working on creating AMIs. This group is referred to as the config-setup group in the remainder of this guide.

AMI creation is never done inside a VPC, so this security group should not be created in a VPC even if the AMI will eventually be used in a VPC.

Note: If a bastion host is being used, then access should be limited to the bastion host security group (in the bastion's region) or to the bastion Elastic IP address (in other regions).

Create a Key Pair

This key pair provides root access to the config-master instance. Create separate key pairs for every config-master instance. Download and store the private key securely.

Warning: The key pair assigned to the config-master instance provides root access, which indirectly provides root access to all Puppet Agent instances that use the configuration master.

Note: If a bastion host is being used, the private key may be deployed only to the bastion and securely backed up.

Create an IAM Role

IAM roles allow for permissions to be assigned to running instances without distributing secret keys to those instances. The IAM role provides access to S3 buckets controlled by configuration roots; however, creating the role is recommended even if you are not planning to connect to a configuration root now. Having the roles gives you the flexibility of adding the policy for connecting to configuration roots in the future.

Roles are specific to accounts, not regions, so all config-master instances in an account (that use the same set of configuration roots) can share IAM roles, regardless of the region they are running in. Any config-master instances that use different sets of configuration roots should have their own IAM roles.

The following is a sample of the policy that should be associated with the IAM role if you are connecting to a configuration root. Replace <config-root name> with the name of an S3 bucket on the configuration root.

```
{
  "Statement": [
    {
      "Action": [
        "s3:Get*",
        "s3:List*"
      ],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:s3:::<config-root name>/*"
      ]
    }
  ]
}
```

If the configuration root is not in the same account the S3 buckets will not show up as resources in the AWS management console. You will need to get the correct name of the S3 bucket from the configuration root's owner. However, as long as you provide the correct bucket name, the config-master instance can access the S3 bucket.

If you are accessing multiple S3 buckets, each must be added to the policy as a resource. You can also create more than one IAM role to control which S3 buckets are used by different config-master instances.

Create an Elastic IP

Each config-master instance requires a public DNS hostname, because they act as HTTP servers and Puppet Agent instances need to be able to consistently access them at the same address. AWS allows you to provide hostnames for EC2 instances through its Elastic IP service.

Create an Elastic IP now. If the config-master instance will be in a VPC, ensure that you choose a VPC-type Elastic IP from the drop-down menu. After launching the instance and associating it with the Elastic IP, you will be able to get the DNS hostname. In the remainder of this document, you should replace the <MasterHost> variable with this hostname.

Depending on your situation you may want to assign a hostname for the master node as part of your DNS setup. The Elastic IP you created can be used for this purpose.

Plan for VPC Topology

The techniques used by the AMIs that will be created later rely on network access to a configuration master instance. Instances running inside a VPC are not automatically able to communicate across VPC subnets. This means that network access must be planned for.

A full description of VPC topologies is beyond the scope of this document. Please refer to AWS documentation for a more complete description of VPC topologies. The basic principle described here is that each VPC subnet (address range and routing information) is tied to a single AWS Availability Zone. In order to provide high-availability solutions multiple Availability Zones (AZs) must be used. This means that a VPC solution for multiple AZs requires multiple subnets, and the AMIs described later are not able to correctly communicate.

The solution to this problem is to run a configuration master in each subnet that will run AMIs. These configuration masters can share application configurations through a common configuration master.

Although information on AutoScale Groups is covered much later, the following information is included here as it relates directly to VPC topologies. It is possible to still have a single Elastic Load-Balancer per region, with multiple AutoScale Groups synchronized to it. Each subnet, or Availability Zone, will require its own AutoScale Group configuration since the User Data must be different.

Create the Config-Master Instance

This section contains instructions for launching and configuring the config-master instance. The initial step is to launch the instance with the following settings:

- Select the Amazon Linux 64-bit AMI.
- The instance type should be chosen based on the number of Puppet Agent instances you expect to use with the config-master instance. If unknown, a Micro instance is probably sufficient for the initial setup. This is particularly true if PAT will not be used except during startup, because Micro instances are burstable.
- Enable Termination Protection. The config-master instance should always be running.
- Select the IAM role created earlier.
- Give the node an appropriate name that will help you distinguish its role (config-master) and specialized purpose, if any.
- Select the key pair created for the config-master instance.
- Select the config-master and config-agent security groups.
- If running inside a VPC (where the master will need to act as a PAT for agents until they obtain a public IP address), disable the Source/Destination Check attribute. This can be disabled once the instance has started.

After launching the instance, associate it with the Elastic IP created for it. After making this association, the public DNS appears in the information about the instance in the AWS management console.

Install Software

Log in to the config-master as the `ec2-user`, using the private key from the key pair. Use `sudo` to get a root shell and do the following:

1. Install (yum install) the following third-party software packages:

```
createrepo
httpd
puppet-server
pinentry
```

2. Copy the following scripts and files from the AWS Essentials SDK into the indicated directories with the indicated permissions (all files should be set to be owned by root:root):

```
enc-aws-tools-(latest).jar in /usr/local/bin with 0644 permissions
config-master-update in /usr/local/bin with 0744 permissions
configure-pat.sh in /usr/local/bin with 0744 permissions
enc-config in /etc/sysconfig with 0644 permissions
site.pp in /etc/puppet/manifests with 0644 permissions
```

3. For configuration masters inside a VPC, configure the PAT to start automatically on boot by adding the following line to /etc/rc.local:

```
/usr/local/bin/configure-pat.sh
```

Set S3 Buckets

If you are using a configuration root, edit /etc/sysconfig/enc-config and add the S3 bucket names you want the config-master instance to synchronize with. The bucket names should be added as a space-separated list. The entire list should be inside quotes.

Generate a GNU Privacy Guard (GPG) Key

A common key is required to sign repositories. The passphrase associated with the private key must be noted. Note that the parameters given here are illustrative, and suitable values for the organization should be used.

Note: Key generation requires entropy, which can take a long time to generate on AWS instances. Entropy generation can be accelerated by causing disk access. To do this, open a separate shell and execute the following commands:

```
sync; sh -c 'echo 3 > /proc/sys/vm/drop_caches'; find /var /usr /lib /srv -type f
-print0 | xargs -0 cat > /dev/null
```

You may need to run the commands several times, until the key generation completes.

1. As root, execute the following command:

```
gpg -q --gen-key # (RSA and RSA (default)), 1024, no expire, <your name>,
<your e-mail address>
```

2. Configure gpg to use the generated key as the default and be set up for automatic signing without asking for the passphrase:

- a. Edit `~/.gnupg/gpg.conf`.
- b. Add `passphrase <phrase>` using the passphrase you entered.

Configure HTTPd

Log in to a root shell and perform the following steps to configure HTTPd:

1. Follow directions in the `/etc/httpd/conf.d/welcome.conf` file for disabling the default Web site.
2. Enable HTTPd to start automatically:

```
chkconfig --level 35 httpd on
```

3. Manually start HTTPd:

```
/etc/init.d/httpd start
```

Configure the Puppet Master Server

Log in to a root shell and perform the following steps to configure Puppet Master:

1. Add the following lines to the top of the `/etc/puppet/puppet.conf` file:

```
certname = <MasterHost>
autosign = true
```

2. Specify the Puppet module path, reversing the order of the two default locations. Edit the `[main]` section of `/etc/puppet/puppet.conf` file to have the following line:

```
modulepath = /usr/share/puppet/modules:$confdir/modules
```

3. Enable Puppet Master to start automatically.

```
chkconfig --level 35 puppetmaster on
```

4. Manually start Puppet Master.

```
/etc/init.d/puppetmaster start
```

Add Required Puppet Modules

There are certain Puppet modules that are universal and installed on all Puppet Masters. The following lines must be executed on the Puppet Master.

```
puppet module install puppetlabs/stdlib
```

The command should indicate that the module is installed in `/usr/share/puppet/modules` based on the configuration changes made above.

Add a Cronjob

The config-master-update script updates applications added to both configuration roots and to the configuration master directly. It should be run regularly through a cronjob. Log in to a root shell and add the config-master-update script to the root crontab (crontab -e) as root. The following contents are typical:

```
0,15,30,45 * * * * /usr/local/bin/config-master-update >> /dev/null 2>&1
```

Note: This line establishes updates every fifteen minutes. For many systems this is sufficient, but more or less frequent updates can easily be configured.

Install ENC-Common

The AWS Essentials SDK includes an application packaged as enc-common.config.tar.gz. It must be installed on either the config-root or config-master before the config-master will work properly. At this point, you should ensure that this application is already installed on the config-root. If it is not, you can request that the config-root owner install it or install it on the config-master directly. Instructions for installing applications can be found in the *Application Installation* chapter of this guide.

Configure the Hostname (Optional)

It may be desirable to set the hostname of the configuration master to make it more identifiable, especially if it will be configured for remote logging. The following commands will set the hostname.

It is recommended that the hostname follow the same pattern as other nodes in the system, meaning that it should include the AWS instance identifier. Using a consistent prefix on the hostname will also allow for easy grouping in remote logging systems, if they are used.

Determine the hostname you want and execute the following in a root shell:

```
HOSTNAME=your desired hostname
DOMAIN=your desired domain (or just site)
LOCALIP=EIP of the configuration master

echo hostname: $HOSTNAME >> /etc/cloud/cloud.cfg
echo fqdn: $HOSTNAME.$DOMAIN >> /etc/cloud/cloud.cfg
echo HOSTNAME=$HOSTNAME >> /etc/sysconfig/network

echo $LOCALIP $HOSTNAME.$DOMAIN $HOSTNAME >> /etc/hosts

/bin/hostname $HOSTNAME
```

Configure Remote Logging (Optional)

Configuration masters are critical application infrastructure and should be guarded and carefully monitored. You will want to ensure that they are secure and operating correctly.

A key aspect of this monitoring is external logging. If you are set up for remote monitoring then it should be configured at this time. As an example, if you are using an external syslog host, you can do the following:

1. Create the `/etc/rsyslog.d/enc-master.conf` file, with the following contents as an example:

```
*.* @syslog-host-name:port
```

2. Restart rsyslog.

```
/etc/init.d/rsyslog restart
```

Final Steps

Log in to a root shell and perform the following steps to finalize the configuration master setup:

1. On configuration masters inside a VPC, execute the following to start the PAT:

```
/usr/local/bin/configure-pat.sh
```

2. Execute the following to update the master with any applications in the configuration root bucket(s) and in the local application directory:

```
/usr/local/bin/config-master-update
```

Conclusion

You have now completed the configuration master setup. The config-master instance should be left running. The next section covers how to create Puppet Agent AMIs to use the configuration master.

Chapter 4: Puppet Agent AMI Creation

The Puppet Agent AMI is a basic image that enables you to launch any number of instances as Puppet Agent nodes. One of the goals of the configuration system is to limit the number of AMIs you need; however, you will sometimes need more than one AMI, because every operating system requires its own AMI. In addition, Puppet Agent nodes that will be running in a VPC have one property that must be configured differently (described in this chapter). This means that you will need more than one AMI if you have some applications running in a VPC and others that are not.

This guide contains instructions for setting up Amazon Linux AMIs. Supporting other operating systems is possible, but is beyond the scope of this document. Please contact PESDCA for guidance.

Note that AMIs are specific to AWS regions, but are not dependent on VPC or non-VPC usage. Create your AMIs outside the VPC.

When launched as an instance, the Puppet Agent AMI is configured to do the following:

- Configure which config-master instance the Puppet Agent instance will use
- Configure external logging
- Install and update operating system software
- Run the Puppet Agent to install and configure applications on the instance

The following sections step through the process required to create an Amazon Linux Puppet Agent AMI.

Launch an Instance

The first step is to launch an instance that will be configured and saved as the Puppet Agent AMI. Choose the following settings:

- Select the Amazon Linux AMI 32- or 64-bit AMI.
- The instance type can be micro for the AMI creation. When you launch instances based on the AMI, you will be able to choose a different type.
- Set User Data to none;none.
- The name is unimportant. You will be destroying this instance after saving it as an AMI.
- Select the key pair created for the config-master instance.
- Select the config-setup security group.

Install Scripts

The AWS Essentials SDK includes a script that needs to be installed and run on each boot of an instance. The necessary files are located in the config-agent directory. Follow these steps to properly install this script:

1. Copy the following scripts and files to a temporary location on the instance:

```
enc-agent-bin
enc-agent-sysconfig
ifup-local
```

2. Log in to the instance as the `ec2-user`, then open a root shell using `sudo`.
3. Rename the scripts and move them to the correct directories.

```
mv enc-agent-sysconfig /etc/sysconfig/enc-agent
mv enc-agent-bin /usr/bin/enc-agent
mv ifup-local /sbin/ifup-local
```

4. Ensure that the scripts have the correct ownership and permissions.

```
chown root:root /etc/sysconfig/enc-agent
chown root:root /usr/bin/enc-agent
chown root:root /sbin/ifup-local
chmod 0644 /etc/sysconfig/enc-agent
chmod 0744 /usr/bin/enc-agent
chmod 0744 /sbin/ifup-local
```

5. Configure the service to start automatically on boot by adding the following line to `/etc/rc.local`:

```
/usr/bin/enc-agent
```

Local and Remote Logging

This section contains important information about local and remote logging and the steps necessary to configure it. Remote logging is a key aspect of this configuration system. Instances may start and be configured at almost any time, and managing the logs across all of the instances would be very difficult without a centralized logging system.

The scripts installed in the previous section configure instances to use Papertrail for log centralization. Papertrail was chosen because of its cost, features, and Web-based access. For small, simple systems, a dedicated, centralized syslog host may be sufficient.

Papertrail creates standard syslog-compatible Log Destinations which are effectively a hostname and port combination, along with a setting that allows new systems to be automatically added. The hostname and port are used as part of the agent configuration.

A full description of using Papertrail is outside the scope of this document.

Papertrail is part of the solution for remote monitoring. In addition to logging, PagerDuty was chosen to provide alerting and incident management. The configuration and management of PagerDuty is outside the scope of this document.

Note: The specific host and port used by Papertrail is included in the `enc-agent` `sysconfig` file. The default values disable remote logging.

Follow these steps to configure remote logging:

1. Edit the `/etc/sysconfig/enc-agent` file.
2. Update the values of `LOGGING_HOST` and `LOGGING_PORT`.
3. Review the other settings and make appropriate changes based on operating system and application needs.
4. Save the file.

Local logging allows local applications to talk to the local syslog server. To enable local logging:

1. Edit the `/etc/rsyslog.conf` file.
2. Uncomment the `$ModLoad` lines for `imudp` and `imtcp`.
3. Uncomment the `$UDPServerRun` line.
4. Uncomment the `$InputTCPServerRun`.
5. Save the file.

Update System Configuration

Several system configuration parameters can be found in the `/etc/sysconfig/enc-agent` file. The previous section discussed some logging configuration.

If the Puppet Agent will run in a VPC, you must set the `IS_PUBLIC_IP_NEEDED` property in this file to true.

Additional parameters can be found in this file and edited appropriately.

There are known issues with encrypted drives that can cause the boot process to fail. This can occur if there are encrypted drives available during boot, even if they are not the boot drive. To prevent this, add the “`rd_NO_LUKS`” option to the kernel boot parameters in the `/etc/grub.conf` file. It should look similar to the following:

```
# created by imagebuilder
default=0
fallback=1
timeout=1
hiddenmenu

title Amazon Linux 2013.03 (3.4.43-43.43.amzn1.x86_64)
root (hd0)
kernel /boot/(image name) root=LABEL=/ console=hvc0 rd_NO_LUKS
initrd /boot/initramfs-3.4.43-43.43.amzn1.x86_64.img
```

Install Puppet Agent

The Puppet Agent and its dependencies must be installed but not run on the base AMI. The Puppet Agent version must match the Puppet Master version installed on the config-master instance.

On an Amazon Linux instance, log in as the `ec2-user`, open a root shell, then install Puppet:

```
yum install puppet
```

Once installed, the `/etc/puppet/puppet.conf` file must be edited. In the `[main]` section, add `"pluginsync = true,"` as shown:

```
[main]
pluginsync = true
-other lines as already present-
```

Disable Services

The Amazon AMI includes several services that require features that are disabled in these instructions. Rather than have error messages printed, these services should be disabled. Execute the following in a root shell:

```
update-motd --disable
```

Configure Repositories

The startup script configures specific repositories when an instance is launched. To prepare for the scripts to do this, all other repositories should be disabled on the AMI.

Log in to the instance as `ec2-user` and open a root shell. Execute the following to disable all repositories and refresh:

```
yum-config-manager --disable '*'
yum clean all
yum makecache
```

You must also disable RPM signature checking. Edit the `/etc/yum.conf` file and set the value of `gpgcheck` to zero.

Final Preparation

As final preparation for saving the AMI, clear log files and histories, and shut the instance down. Log in to the instance as `ec2-user` in a root shell, and follow these steps:

1. Clear logs and histories.

```
/etc/init.d/rsyslog stop
find /var/log -type f -delete
rm -f ~ec2-user/.bash_history
rm -f ~/.bash_history
```

2. Shut the instance down.

```
history -c && shutdown -h now
```

Create the AMI

After the machine has been shut down, use the AWS API or management console to create an Elastic Block Store (EBS) AMI image. Give the AMI a name that will help you remember which base operating system it uses. You may also wish to include some type of versioning information. For example, “base-amzn-2012-09-x86_64-1” might be a good image name. If the Puppet Agent AMI is for nodes that will run in a VPC, adding vpc to the AMI name is recommended.

When you create an AMI, a globally unique identifier that begins with AMI is assigned. This identifier is used to choose a specific AMI when command-line tools are used.

After creating the AMI from the instance, terminate the EC2 instance.

Using the AMI

The AMI just created allows for instances to be easily started and configured by the configuration master. The scripts that were installed place certain requirements on the instance that will cause failures if not met. These requirements are described in the following sections.

User Data

The instances must be configured with the instance type and hostname of the configuration master they should use. These two strings must be passed to the instance through the user data string, separated by a semi-colon character. For example:

```
config-validation;host.name.of.config.master
```

When the same AMI is used in different VPC subnets there may not be a single configuration master available. In this case additional information can be specified as shown here.

```
config-validation;subnet-ID1=host-for-subnet-1,subnet-ID2=host-for-subnet-2
```

The subnet IDs come directly for the AWS subnets.

IAM Role

Instances must be able to read tags when launched to properly configure themselves. This means all must be assigned an IAM role with at least the EC2 DescribeTags permission granted. All of your instances that use these AMIs must be launched with an IAM role. Most applications that use this AMI will require other IAM permissions and will need to define an IAM role that includes those permissions in addition to the DescribeTags permission.

If you do not have an existing IAM role, create one now to use for validation (described later) with the following minimal policy:

```
{
  "Statement": [
```

```

    {
      "Action": [
        "ec2:DescribeTags"
      ],
      "Effect": "Allow",
      "Resource": [
        "*"
      ]
    }
  ]
}

```

Updating an AMI

Existing AMIs can be updated by launching an instance from the AMI, making appropriate changes, and then creating a new AMI.

Note: Launch the instance with User Data of none;none. This prevents the AMI from customizing itself.

After making required changes, follow the steps in the *Final Preparation and Create the AMI* sections above. Modify the name of the AMI to indicate the new version.

Any Auto Scaling Groups that refer to the old AMI (by AMI ID) should be updated to refer to the new AMI.

Chapter 5: Application Installation

It is common for an application to require a combination of Puppet scripts, files, and RPMs. Rather than manage these files separately, it is desirable to bundle the files together (in a .tar.gz file) and install them together. This bundle is referred to as an Application Configuration Archive (ACA).

ACAs can be installed in configuration roots to be globally shared or directly on a configuration master (in which case only Puppet Agent instances associated with the master have access to the software).

You should ensure that you have a high level of trust for an application provider before installing their applications. If you have any doubt about the application provider, but still want to try their applications, you may wish to set up a dedicated S3 bucket on a configuration root or a dedicated configuration master (if you are not using a configuration root) to store their applications separately.

Configuration Root Installation

The following steps install an ACA into a configuration root's S3 bucket:

1. Ensure there is a directory on the config-root instance that represents the S3 bucket where you want to install the ACA.
2. Copy the ACA to this bucket directory.
3. Generate an index file of the bucket contents. This is used to optimize synchronization of bucket contents. In the directory that represents the bucket, execute the following:

```
rm -f index; sha256sum -b * > index
```

4. Using the S3 command-line client, update the contents of the S3 bucket with the files in the config-root directory. Update the index file last. Ensure that the correct access permissions are retained or created for new files.

Configuration Master Installation

The following steps install an ACA on a config-master instance:

1. Copy the ACA to the config-master instance, placing it in the */var/lib/enc-config/local* directory. You may need to create this directory the first time you install an application.
2. Log in to the config-master instance, use `sudo` to obtain a root shell, and execute the `config-master-update` script.

```
/usr/local/bin/config-master-update
```

Note: You can install several ACAs at once by repeating step one for each before running the update script.

Chapter 6: System Validation

After installing a configuration system, including the `enc-common.config.tar.gz` ACA, you can validate that the system is operating correctly using the instructions in this chapter.

This process must take place in an AWS account and region that contain a `config-master` instance.

1. Launch an instance from a Puppet Agent AMI. The instance should use the following configuration:
 - Set User Data to `config-validation;<MasterHost>`.
 - Assign an IAM role with at least the `EC2 DescribeTags` permission.
 - Select an appropriate key pair (although the node will be shut down immediately with no need to log in).
 - Select both the `config-agent` and `config-setup` Security Groups.
2. Log in to the `config-master` instance, open a root shell, and create an override by doing the following:
 - a. Create directories and files.

```
mkdir -p /var/lib/enc-config/local/nodes
cd /var/lib/enc-config/local
cp /etc/puppet/manifests/nodes/config-validation.pp nodes/
```

- b. Edit the `config-validation.pp` file in the location where you just copied it and insert the text “`OVERRIDDEN`” in the message before “`config-validation.`”
 - c. Update the master files to include the overrides.

```
/usr/local/bin/config-master-update
```

3. Using AWS, launch another instance using the same process as before. This instance will see the overridden definition.
4. To validate the system operated correctly, check the logs and verify that two instances launched and logged different messages.
5. Remove the override from the `config-master` instance.

```
rm /var/lib/enc-config/local/nodes/config-validation.pp
/usr/local/bin/config-master-update
```

6. Use AWS to terminate the test instances.

Note: It is possible to use the same AWS account to host multiple `root/master/instance` combinations. Care must be taken to be consistent in naming, particularly for items like Security Groups and IAM configurations. This way a new system can be installed and validated in parallel with an existing system.

Chapter 7: Application Developer Information

This section contains information to help developers create and package ACAs that can be installed on AWS Essentials application servers and configuration systems. The information in this chapter assumes you have access to your own configuration system for testing purposes. You may need to follow instructions in the previous chapters to set up a development system if you do not already have one.

Application Creation

It is typical for an ACA to bundle Puppet configuration along with RPMs and other files. Application providers need to create these files and bundle them in an appropriate directory structure.

Puppet relies on two constructs to create applications:

- Manifests
- Modules

A full description of Puppet is beyond the scope of this document, but some important concepts and their support are mentioned.

The system is built to support two Puppet directories: nodes and modules. The nodes directory, which is a subdirectory of the Puppet manifests directory, defines nodes, which in turn are regular expressions that match instance hostnames. The modules directory contains Puppet module definitions, which is where the application requirements are defined. These directories can be stored in configuration root buckets and then deployed to the configuration master using scheduled script execution. They can also be stored directly on the configuration master.

As an example, here are several files that define a very simple application. The first file is *site.pp*, which is placed at the root of the manifests directory. This file is included with the AWS Essentials SDK and should already be present.

```
# This file is not typically used to define nodes, rather
# they are defined in separate files that are imported.
# NOTE: Puppet does not monitor imported files, so any change
# to the directory or the files requires that Puppet reload.
# Reloading can be accomplished by 'touching' this file or
# restarting the Puppet service.
#
import 'nodes/*.pp'

node default {
}
```

Next, a template for nodes is created. This file should be given a name appropriate for the application and a .pp extension. It should be placed in a manifests/nodes directory. The following is an example template from the config-validation program included in the enc-common.config.tar.gz ACA.

```
node /config-validation-*/ {
  notify { 'The config-validation step was matched and executed.': }
}
```

With these two files in place, a node with User Data “config-validation” and an appropriate config-master hostname should boot up and output the specified message in its log.

Any application will require more configuration than this simple example provides. This is done by adding directories of configuration files to the modules directory. The Puppet documentation contains more information on creating modules, but the general rules are:

1. Create a directory under modules that is name of the module.
2. Create a directory under modules that is name of the module.
3. Create a directory called manifests in that directory.
4. Create a file called init.pp that contains a class with the same name as the module.

For example, here is a module called “enc” that defines the installation of a JRE. The directory structure looks like this (all in the modules directory):

```
enc
enc/manifests
enc/manifests/init.pp
enc/manifests/jre.pp
```

The init.pp file contains an empty class, but this is required.

```
class enc {
}
```

The jre.pp file contains the instructions to install the JRE that are part of the module.

```
class enc::jre {
  package { 'jre':
    ensure => '1.6.0_35-fcs',
  }
}
```

Other .pp files can be created to define other requirements. With this module, a node definition (in a file under the nodes directory) can install the JRE as follows:

```
node /hostname-that-needs-jre-*/ {
  include enc::jre
}
```

This section only briefly touches on Puppet application definition. More information can be found in the Puppet documentation.

Note: To keep existing instances stable, all Puppet configurations should use explicit versioning. For example, a given system (set of instances) should “lock in” on a particular application version and only change if the configuration files are changed. This means that Puppet manifests and module names should almost always be version specific.

A more complete application would include not only the Puppet manifests and modules, but other (Puppet) files and RPMs. Applications are distributed as archive files (.tar) and compressed (.gz). The structure of this archive is simple, with each type of file placed in a different top-level directory.

Note: If an ACA is installed directly on a config-master instance, your .tar file will be expanded in the /var/lib/enc-config/local/ directory and directories will be merged with directories of the same name in that location.

Your application archive may contain the following directories:

- **repos.** This directory maps to the software repositories. Subdirectories will be available on the configuration master node through http. Directories called repodata will automatically have createrepo and signature commands executed on the master node.
- **nodes.** This directory maps to the Puppet manifests/nodes directory. Files in this directory are merged and define the types of instances that can be configured.
- **modules.** This directory maps to the Puppet modules directory, as discussed above.

After you create this directory structure and place application files in it appropriately, use a simple tar command to create an ACA that can be installed on the configuration system. It is desirable that the ACA name contain the name and version of the application.

```
mkdir -p <app>/nodes
mkdir -p <app>/modules
mkdir -p <app>/repos
tar czfC <application>-<version>.config.tar.gz <app> .
```

Application Override Management

This section contains information that will help you manage your own config-master instance during development for testing purposes. During application development, it is often not worth the effort to update a configuration root for every minor change and before running every minor test. It may be beneficial to download some shared applications and configuration from a configuration root, but override working application files at the configuration master level.

In any configuration system, the following process takes place when a configuration master is updated:

1. ACAs are downloaded from configuration root buckets if the configuration master is configured to synchronize with them.
2. ACAs in the config-master instance's `/var/lib/enc-config/local` directory are expanded. If any of the files in these archives match filenames downloaded from root buckets, they overwrite the previously downloaded files.
3. Files stored in subdirectories of `/var/lib/enc-config/local` are then copied into the directory structure. These files again overwrite any existing files with the same names (from root buckets or ACAs).

Because of this process, you can provide temporary overrides during the development process:

- You can override an entire ACA from the configuration root by storing a new ACA with the same files in it at `/var/lib/enc-config/local`.
- You can override individual files in an ACA by storing them in the following subdirectories at `/var/lib/enc-config/local`:
 - **repos.** This directory contains repositories.
 - **nodes and modules.** These directories contain Puppet files, as described earlier.

To force the inclusion of overridden archives or files, manually execute the `config-master-update` script on the config-master instance then launch a new Puppet Agent instance. This override process enables developers to quickly test changes, and then create new ACAs that can be installed on the configuration system.

Chapter 8: Conclusion

The goal of this document is to detail a solution to automated configuration that includes best practices for Amazon. This is not the only method that is possible, and the methods detailed here are not meant to be exclusive to other methods.

These techniques provide for an extremely scalable solution that also allows for secure instances.

Chapter 9: Appendix A: Glossary

The following terms are used in this guide:

Account

An account is the most basic AWS component. In order to use Amazon services an account is required. When this term is used it represents the highest level of access. This is equivalent to the Unix root account, but on the scale of an entire application platform.

Access to an account is allowed through the web management console and through AWS APIs (using the account credentials). Security implications of access to the account are discussed throughout this guide.

Region

AWS is divided by geographic region, and further divided by availability zones. Regions operate as separate entities, and many AWS features do not operate across regions. Any application deployment plan needs to address regional and availability zone differences.

User

A user represents limited account access, either through AWS APIs or through a web console. This limited access is accomplished by using credentials that are created for the user or program, and thus the user access can be restricted.

Amazon Machine Instance

An Amazon Machine Instance, or AMI, represents a disk image that is used to bootstrap an instance. Pre-built AMIs are available for use or an account can create its own.

Each AMI is tied to a specific AWS region, meaning that each Amazon region must use AMIs dedicated to that region.

Instance

An instance represents a running virtual machine. An instance has a unique identifier (instance ID), and is associated with a set of resources (processor, memory) and a disk for storage.

Access to the instance is key aspect of security. The ability to create new instances is also a security issue, and represents the ability to add cost to a platform as each instance is paid for based on the time it is running. Security information may also be copied to instances, and instances may have AWS access based on roles. Because of this, access to a console of an instance and the ability to create new instances needs to be carefully guarded.

Auto Scaling Group

An Auto Scaling Group represents a monitored set of instances of the same type. Monitoring the group allows for a minimum number of instances to always be running (providing redundancy) and allows a flexible number of total instances (providing scalability).

Creating a fault-tolerant system relies heavily on the use of Auto Scaling Groups for application management. Auto Scaling Groups are a good way of restricting security access, since the definition of an Auto Scaling Group can be made by an administrator, with only the ability to start and stop machines delegated to other users.

Chapter 10: Appendix B: Bastion Hosts

The use of a bastion host is recommended for highly secure systems. A bastion host is a single-point-of-entry for an application or account. The key principles of a bastion host are the following:

- Shell access to all other instances must come only from the bastion.
- The bastion uses individual user accounts for those who should have access, and uses user-controlled SSH to access those accounts.
- All access is keylogged, and those logs are protected against deletion. This provides a secure audit-trail for all interactions with the instances.
- The bastion may provide role-based access to AWS services, mitigating the need to assign read-only IAM accounts.
- Common management tools can be deployed to the bastion and become available to all users.
- The bastion host limits SSH access appropriately, which may mean using VPC services or appropriate firewall settings.

The specific setup of a bastion host is not covered in this document, but the implications of using one and its effect on other configuration is noted.

A typical process for a bastion setup would include:

- Starting an appropriate instance, running a secure OS.
- Configuring appropriate remote logging. This includes installation of a key logger so that all user keystrokes are logged.
- Assigning an elastic IP address to the host so that other machines in different regions can lock SSH access to that IP.
- Creating a security group for SSH access in the region of the bastion. This group would include all instances in the region and is used to lock SSH access to the security group. This is required because intra-region IP addresses are dynamic.
- Creating individual user accounts on the bastion, and configuring SSH access for those users. A recommendation is to leave password logins disabled and install users' generated public keys on the bastion host.

The use of a bastion host has minimal, but important, impacts on the configurations described in this document. These are noted where appropriate.

Chapter 11: Appendix C: Auto Scaling Groups

One of the important benefits of this system is the ease of incorporating auto scaling groups. Auto scaling groups are an AWS feature that unifies monitoring and instance management based on a set of rules and schedules.

Using auto scaling groups usually means creating dedicated AMIs for each type of instance. This can be tedious, error prone, and the work does not transfer between AWS regions.

By comparison, the techniques described in this document simplify the AMI requirements because only a single base AMI is required per platform. An auto scaling group can set the user data of the instances that it creates, and this is enough (using the techniques of this document) to create any type of instance.

A complete description of auto scaling groups is beyond the scope of this document. The following sections discuss the specific impacts of auto scaling groups when used in combination with the other techniques discussed in this document.

Creating the Launch Configuration

Auto scaling groups are governed by a launch configuration. The launch configuration describes the instances that will be created by the auto scaling group.

The following is an example of creating a launch configuration in the Singapore region based on one particular account setup, and will not run in other accounts. It uses the command-line tools provided by Amazon that are described at <http://aws.amazon.com/developertools/2535>.

```
./as-create-launch-config.cmd ttc-25-auth --region ap-southeast-1 --image-id ami-529ddd00 --instance-type m1.small --user-data-file userdata --group config-agent,ttc-25-internal --key root-ttc-25-ap-southeast --iam-instance-profile ttc-25-auth
```

This particular command creates a scale group called 'ttc-25-auth' in the Singapore region. The other options are critical and described here:

- `--image-id`: This specifies the AMI that will be used for all instances in the auto scaling group. This should correspond to a base AMI created as described earlier in this document.
- `--instance-type`: This specifies the size of instance that will be created.
- `--user-data-file`: The input to user data described in this document uses a semi-colon to separate the machine type from the configuration master addresses (MasterHost). Because of this choice it is not possible to pass the user data on the command line, rather the appropriate data must be put in a file and the 'user-data-file' option used.

- `--group`: All instances must be placed in the 'config-agent' security group to allow communication with the configuration master. Additional groups would also be specified based on the instance type.
- `--key`: The key specified depends on the type of the instance. This key is not used by the configuration infrastructure.
- `-iam-instance-profile`: The profile selected also depends on the type of the instance. The profile is not used by the configuration infrastructure.

If creating the launch configuration is successful then it can be used to create auto scaling groups.

Creating the Auto Scaling Group

Creating an auto scaling group requires that a launch configuration exist. There are no direct relationships between the auto scaling groups and the configuration infrastructure, the following command is shown only as an example of the options.

```
./as-create-auto-scaling-group.cmd ttc-25-auth --region ap-southeast-1 --
    availability-zones ap-southeast-1a --launch-configuration ttc-25-auth --
    max-size 1 --min-size 1
```

Once created, the auto scaling group will create instances as required and based on the specified launch configuration.

Although not shown in this example, it is common for an auto scaling group to be related to a load balancer. This is done with the '`--load-balancers`' option, which specifies a list of load balancers to keep synchronized with the auto scaling group.

Auto Scaling Features

Auto scaling groups allow for a variety of control and monitoring. The following suggestions highlight some of the more important actions that should be considered with auto scaling groups:

- Monitoring for failures, both creating and destroying instances. These failures can trigger AWS SNS messages, which can in-turn be used with alerts.
- Automatic scaling based on load can be enabled easily using AWS CloudWatch. This allows scaling up or down based on high or low load.
- Utilizing the load-balancer health check to terminate instances.

Chapter 12: Appendix D: AWS Essentials Management

This chapter contains information useful for understanding AWS Essentials and for managing it both as an application server and a development platform.

Best Practices

A goal of this document is to present best practices for AWS. Each group that uses AWS has its own idea of what “best practices” are, and so as a guide the following goals are specified as requirements:

- Security is critical. All permissions and security relationships should be understood and documented. Processes for password/key aging should be part of the platform. Least permissions is the guiding principle.
- AMIs should be used minimally. Because AMIs cannot move between regions, any effort placed on customizing AMIs is effort that must be duplicated for a worldwide platform.
- All software updates should be managed. There should be no automatic updates of software that are not approved.
- Automation is important, and processes that are scriptable are chosen over those that require GUIs or other manual intervention.
- Existing services should be used where possible.

Instance Overview

The processes and scripts documented here implement a general sequence of steps that are common to all instances. There is little or no modification allowed to this core sequence, with customization occurring after this sequence is completed.

The sequence presented has a goal of getting an instance into a managed state as quickly as possible. This means that the instance is uniquely identified and logging remotely, so that any problems can be identified through remote monitoring. This means that instance failures do not automatically mean loss of diagnostic and debugging information available in the remote log.

As described in this document, each instance that is launched has a purpose, and that purpose is identified as the “type” of the instance. This should not be confused with the AWS meaning of instance type (for example, small, medium, large, etc.). This document will use “type” as an indication of the applications use of the instance. For example, “web-server”, “database”, and “key-server”, could all be types.

An AWS instance is “born” with very limited configuration:

- It has an instance ID, which is globally unique. AWS has stated: “Instance ids are unique. You'll never receive a duplicate id. However, the current format of the

instance id is an implementation detail that is subject to change. If you use the instance id as a string, you should be fine.”

- It has a user data field that contains information passed in when the instance is started. The user data field is part of the auto scaling group configuration and applies to all instances started in that group. This means that it should not contain instance-specific configuration, rather it should contain group-specific information.
- It is running with a defined set of resources, including CPU, memory, and disk storage. It also is started from a given AMI.
- It has some configured network access, also based on how it was started.

Based on this initial information, the following sequence of steps is run on every instance.

1. The local hostname of the instance is set. This follows the format of ‘type-instance id’.
2. Remote logging is configured.
3. An update of system software components is performed from a private repository.
4. A Puppet client call is made that begins the process of application deployment.

This minimal process ensures a consistent baseline for application installation. It also ensures unique identification and remote logging. A minimal amount of configuration is required, as follows:

- The instance ID, which is generated by AWS and available to the instance.
- An application instance type, which is determined by the user and available to the instance through the user data.
- The location of a remote logging service, which is also encoded into the AMI.
- The location of a configuration master server, which is determined through user data.

Note that there are only two sources of configuration: the AMI and the AWS instance data (which includes the instance and user data). Applications are free to use other sources of information, this document is only describing the minimum required configuration.

Type Guidelines

The instance hostname is formed from a type name along with the instance identifier. The use of Puppet provides some guidelines when choosing the type names that are presented here.

Puppet matches instances to manifests based on the hostname. The matching process can utilize regular expression matching, which is required in the dynamic system described here. The ‘instance’ portion of the hostname is easily matched by an ending wildcard.

In general, the use of regular expressions works well with a hierarchical naming scheme, starting with the most generic and getting progressively more detailed. For example: `systemtype-systemid-application_type`.

Application Responsibilities

The purpose of AWS instances is to run applications. There must be some relationship between the instance and the applications that it runs in order for the instances to operate correctly.

Logging

The most important application requirement is to correctly log and use appropriate levels (warn, error, etc.). The instance configuration centralizes logs, and in most systems these logs are monitored for errors that will cause alerts to be sent.

The default instance configuration described in this document uses syslog to centralize logs, and so applications should ensure that their logs make it to the instance syslog handler.

Startup

The default instance startup assumes that if the instance continues to run then it is running correctly. This means that fatal errors at the application level should cause the instance to (eventually) shut down.

In the case where the instance is running servers that are hooked to AWS load balancers, the application should stop these servers if non-recoverable errors occur. This will also cause the AWS monitoring to terminate the instance.

Chapter 13: Glossary

M

My Term

My definition